

**ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN**

—o0o—



ĐỒ ÁN CHUYÊN ĐỀ TỔNG HỢP

**XÂY DỰNG HỆ THỐNG XÁC THỰC TẬP
TRUNG
DỰA TRÊN KEYCLOAK VỚI QUY TRÌNH
TRIỂN KHAI TỰ ĐỘNG**

NGÀNH: CÔNG NGHỆ THÔNG TIN

NGUYỄN TRỌNG HÒA

HÀ NỘI - 2026

ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

—o0o—



ĐỒ ÁN CHUYÊN ĐỀ TỔNG HỢP

**XÂY DỰNG HỆ THỐNG XÁC THỰC TẬP TRUNG DỰA
TRÊN KEYCLOAK
VỚI QUY TRÌNH TRIỂN KHAI TỰ ĐỘNG**

NGÀNH: CÔNG NGHỆ THÔNG TIN

Sinh viên thực hiện: NGUYỄN TRỌNG HÒA
Lớp 68MHT2 - MSSV 0011468

Giảng viên hướng dẫn: ThS. NGUYỄN VIỆT NHẬT

HÀ NỘI - 2026

LỜI CAM ĐOAN

Em xin cam đoan nội dung của báo cáo chuyên đề này là kết quả do chính em thực hiện dưới sự hướng dẫn của ThS. Nguyễn Việt Nhật. Các nội dung trình bày trong báo cáo, bao gồm phần phân tích, thiết kế, triển khai và đánh giá hệ thống, đều do em trực tiếp thực hiện, không sao chép từ bất kỳ công trình nào khác. Mọi tài liệu tham khảo, số liệu và kết quả của tác giả khác được sử dụng trong báo cáo đều đã được trích dẫn và ghi rõ nguồn theo đúng quy định.

Nếu có bất kỳ sự gian lận nào trong nội dung học thuật, em xin hoàn toàn chịu trách nhiệm trước Khoa Công nghệ thông tin và Nhà trường.

Hà Nội, ngày tháng năm 2026

Sinh viên thực hiện

NGUYỄN TRỌNG HÒA

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo chuyên đề, em đã nhận được rất nhiều sự giúp đỡ, hướng dẫn và động viên quý báu.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Nguyễn Việt Nhật – giảng viên hướng dẫn – người đã định hướng đề tài, góp ý về kiến trúc hệ thống và tận tình chỉ bảo trong suốt quá trình em thực hiện.

Em xin chân thành cảm ơn các thầy cô trong Bộ môn Kỹ thuật Hệ thống, Khoa Công nghệ thông tin, Trường Đại học Xây dựng Hà Nội đã truyền đạt những kiến thức nền tảng về hệ thống mạng, bảo mật và hạ tầng – những cơ sở quan trọng để em hoàn thành chuyên đề này.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, động viên em trong quá trình học tập và nghiên cứu.

Do kiến thức và thời gian có hạn, báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy cô và các bạn để đề tài được hoàn thiện hơn.

Hà Nội, ngày tháng năm 2026

Sinh viên thực hiện

NGUYỄN TRỌNG HÒA

Mục lục

LỜI CAM ĐOAN	1
LỜI CẢM ƠN	2
DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ	6
ĐẶT VẤN ĐỀ	10
1 TỔNG QUAN VỀ XÁC THỰC TẬP TRUNG VÀ TRIỂN KHAI TỰ ĐỘNG	13
1.1 Xác thực, định danh và quản lý truy cập	13
1.1.1 Khái niệm xác thực và ủy quyền	13
1.1.2 Đăng nhập một lần và định danh liên kết	13
1.1.3 Các mối đe dọa phổ biến đối với hệ thống xác thực	13
1.1.4 Lưu trữ thông tin xác thực an toàn	14
1.2 Giao thức OAuth 2.0 và OpenID Connect	14
1.2.1 OAuth 2.0	14
1.2.2 OpenID Connect	14
1.2.3 Luồng Authorization Code và PKCE	15
1.2.4 Cấu trúc JWT và xác thực token	15
1.2.5 Các loại token và vòng đời	15
1.2.6 So sánh các luồng xác thực OIDC	15
1.3 Giải pháp xác thực tập trung: Keycloak	16
1.3.1 Giới thiệu Keycloak	16
1.3.2 Các khái niệm chính	16
1.3.3 So sánh với một số giải pháp khác	16
1.3.4 Kiến trúc nội bộ của Keycloak	16
1.3.5 Các chuẩn và giao thức liên quan	17
1.4 Ảo hóa container và Kubernetes	17
1.4.1 Container hóa với Docker	17
1.4.2 Kubernetes và k3s	18
1.4.3 Helm	18
1.4.4 Các đối tượng cốt lõi của Kubernetes	18
1.4.5 Container hoạt động như thế nào	18
1.4.6 Kiến trúc của Kubernetes	19

1.5	Triển khai tự động	19
1.5.1	Tích hợp và triển khai liên tục (CI/CD)	19
1.5.2	Hạ tầng dưới dạng mã (IaC)	19
1.5.3	GitOps và ArgoCD	19
1.5.4	Quản lý cấu hình như mã nguồn	19
1.5.5	Lợi ích và thách thức của triển khai tự động	19
1.5.6	Ví dụ một quy trình CI/CD điển hình	20
1.6	Kết chương	20
2	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	21
2.1	Phân tích yêu cầu	21
2.1.1	Yêu cầu chức năng	21
2.1.2	Yêu cầu phi chức năng	21
2.1.3	Tác nhân và ca sử dụng	21
2.2	Kiến trúc tổng thể	22
2.2.1	Mô hình kiến trúc C4	22
2.3	Thiết kế luồng xác thực OIDC	23
2.4	Thiết kế quản lý cấu hình định danh như mã nguồn	23
2.5	Thiết kế quy trình triển khai tự động	24
2.6	Thiết kế cơ sở dữ liệu và trạng thái phiên	25
2.7	Thiết kế tính sẵn sàng cao	25
2.8	Thiết kế bảo mật	25
2.9	Thiết kế cấu trúc kho mã nguồn	26
2.10	Thiết kế quản lý bí mật	26
2.11	Thiết kế thành phần hệ thống	26
2.12	Kết chương	26
3	TRIỂN KHAI VÀ CẤU HÌNH HỆ THỐNG	27
3.1	Môi trường và công cụ	27
3.2	Triển khai Keycloak và PostgreSQL bằng Docker Compose	27
3.2.1	Kiểm tra và xác thực dịch vụ	28
3.3	Cấu hình định danh như mã nguồn	29
3.3.1	Áp dụng cấu hình tự động với keycloak-config-cli	29
3.4	Ứng dụng demo và đăng nhập SSO	30
3.4.1	Mã nguồn ứng dụng demo	30
3.5	Đóng gói và triển khai trên Kubernetes	31
3.5.1	Manifest triển khai Keycloak	32
3.6	Tự động hóa bằng CI/CD và GitOps	34

3.7	Giám sát và vận hành	35
3.8	Quản lý TLS với cert-manager	35
3.9	Quản lý bí mật và cấu trúc kho GitOps	36
3.10	Quy trình triển khai từng bước	37
3.11	Kiểm thử khả năng tự phục hồi	37
3.12	Triển khai cơ sở dữ liệu và bảo mật mạng	37
3.13	Kết chương	38
4	ĐÁNH GIÁ KẾT QUẢ VÀ THẢO LUẬN	40
4.1	Phương pháp đánh giá	40
4.2	Kết quả kiểm thử chức năng	40
4.3	Đánh giá bảo mật	40
4.4	Đánh giá mức độ tự động hóa	41
4.5	Đánh giá hiệu năng	42
4.6	So sánh với phương án triển khai thử công	42
4.7	Thảo luận	42
4.8	Kết chương	43
	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	44
	TÀI LIỆU THAM KHẢO	46
	PHỤ LỤC	47

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

STT	Từ viết tắt	Giải nghĩa
1	API	Application Programming Interface – Giao diện lập trình ứng dụng
2	CI/CD	Continuous Integration / Continuous Deployment – Tích hợp và triển khai liên tục
3	CSDL	Cơ sở dữ liệu
4	GitOps	Phương pháp vận hành hệ thống lấy Git làm nguồn cấu hình duy nhất
5	HA	High Availability – Tính sẵn sàng cao
6	HTTP	Hypertext Transfer Protocol – Giao thức truyền tải siêu văn bản
7	IaC	Infrastructure as Code – Hạ tầng dưới dạng mã
8	IdP	Identity Provider – Nhà cung cấp định danh
9	IAM	Identity and Access Management – Quản lý định danh và truy cập
10	JWT	JSON Web Token
11	K8s	Kubernetes – Nền tảng điều phối container
12	MFA	Multi-Factor Authentication – Xác thực đa yếu tố
13	OIDC	OpenID Connect
14	OAuth	Open Authorization
15	PKCE	Proof Key for Code Exchange
16	RP	Relying Party – Bên tin cậy (ứng dụng dựa vào IdP)
17	SSO	Single Sign-On – Đăng nhập một lần
18	TLS	Transport Layer Security – Giao thức bảo mật tầng truyền tải

STT	Từ viết tắt	Giải nghĩa
19	TOTP	Time-based One-Time Password – Mật khẩu dùng một lần theo thời gian
20	RBAC	Role-Based Access Control – Kiểm soát truy cập theo vai trò

Ghi chú: Danh mục được sắp xếp theo thứ tự bảng chữ cái của từ viết tắt. Cập nhật thêm khi bổ sung thuật ngữ mới trong quá trình viết.

Danh sách hình vẽ

2.1	Kiến trúc tổng thể ba tầng của hệ thống	22
2.2	Sơ đồ container của hệ thống (mức C4)	23

Danh sách bảng

2	Kế hoạch thực hiện chuyên đề theo giai đoạn	12
1.1	So sánh các luồng xác thực OIDC	16
1.2	So sánh Keycloak với một số giải pháp định danh	17
2.1	Danh sách yêu cầu chức năng	21
2.2	Danh sách yêu cầu phi chức năng	21
2.3	Tác nhân và ca sử dụng chính	22
2.4	Các bước của luồng Authorization Code kèm PKCE	24
2.5	Các nhóm dữ liệu chính trong hệ thống	25
2.6	Ảnh xạ mối đe dọa và biện pháp thiết kế	25
2.7	Cấu trúc thư mục kho mã nguồn dự án	26
3.1	Các công cụ và công nghệ sử dụng	27
3.2	Kết quả kiểm thử tự phục hồi (khung ghi nhận)	37
4.1	Kết quả kiểm thử chức năng	40
4.2	Đối chiếu yêu cầu chức năng và kết quả	41
4.3	Phân tích mối đe dọa theo STRIDE	41
4.4	Mức độ tự động hóa đạt được	41
4.5	So sánh triển khai thủ công và tự động	42
4.6	Ưu điểm và hạn chế của hệ thống	42
4.7	Đối chiếu mục tiêu và kết quả đạt được	44

ĐẶT VẤN ĐỀ

1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số, một tổ chức thường vận hành đồng thời nhiều ứng dụng và dịch vụ khác nhau: thư điện tử, hệ thống quản trị nội bộ, cổng thông tin, các ứng dụng nghiệp vụ, dịch vụ đám mây, v.v. Nếu mỗi ứng dụng tự quản lý tài khoản riêng, người dùng phải ghi nhớ nhiều bộ tài khoản/mật khẩu, dẫn tới việc tái sử dụng mật khẩu, đặt mật khẩu yếu và gia tăng nguy cơ bị tấn công. Về phía quản trị, việc cấp phát, thu hồi và đồng bộ tài khoản thủ công trên nhiều hệ thống vừa tốn thời gian, vừa khó kiểm soát và dễ sai sót.

Một hệ thống xác thực tập trung (xây dựng quanh một *Identity Provider* – IdP) cho phép nhiều ứng dụng cùng tin cậy vào một nguồn định danh duy nhất, hỗ trợ đăng nhập một lần (Single Sign-On – SSO), tập trung hóa chính sách bảo mật như xác thực đa yếu tố (MFA) và cung cấp một điểm kiểm soát, giám sát duy nhất. Đây là mô hình đã được các tổ chức lớn áp dụng rộng rãi.

Song song đó, bản thân dịch vụ xác thực là một dịch vụ có tính trọng yếu (trust-critical): nó có trạng thái (cơ sở dữ liệu, phiên), yêu cầu bảo mật cao và là điểm truy cập bắt buộc của mọi ứng dụng. Việc triển khai và vận hành dịch vụ này một cách thủ công dễ dẫn tới sai sót cấu hình, khó tái lập môi trường, thời gian khắc phục sự cố dài và khó mở rộng. Do đó, tự động hóa quá trình triển khai và vận hành dịch vụ xác thực là một bài toán có ý nghĩa thực tiễn rõ ràng.

Từ hai nhận định trên, chuyên đề lựa chọn hướng tiếp cận kết hợp: xây dựng một hệ thống xác thực tập trung dựa trên Keycloak, đồng thời nghiên cứu và xây dựng quy trình triển khai tự động cho chính dịch vụ đó.

2. Mục tiêu của chuyên đề

Chuyên đề hướng tới các mục tiêu sau:

- **Mục tiêu 1:** Nghiên cứu cơ sở lý thuyết về xác thực tập trung và các giao thức liên quan (OAuth 2.0, OpenID Connect), từ đó thiết kế một hệ thống định danh tập trung hỗ trợ SSO dựa trên Keycloak.
- **Mục tiêu 2:** Xây dựng quy trình quản lý cấu hình định danh như mã nguồn (Configuration-as-Code) và tự động hóa việc triển khai dịch vụ bằng các công cụ CI/CD, IaC và GitOps.

- **Mục tiêu 3:** Đánh giá hệ thống ở các khía cạnh chức năng, khả năng tự động hóa và bảo mật; đề xuất hướng phát triển thành đồ án tốt nghiệp.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: các giao thức xác thực/ủy quyền (OAuth 2.0, OIDC), nền tảng Keycloak, công nghệ container hóa và các phương pháp triển khai tự động.

Phạm vi nghiên cứu: chuyên đề tập trung vào:

- Xây dựng IdP tập trung với Keycloak, PostgreSQL và một ứng dụng demo đóng vai trò Relying Party sử dụng luồng *Authorization Code* kèm PKCE.
- Quản lý cấu hình định danh (realm, client, role, chính sách MFA) dưới dạng mã nguồn.
- Thiết kế và bước đầu hiện thực quy trình triển khai tự động trên nền tảng Kubernetes (k3s) theo mô hình GitOps.

Trong khuôn khổ thời gian của chuyên đề, một số hạng mục nâng cao (đa vùng, cân bằng tải nâng cao, kiểm thử hiệu năng quy mô lớn) được trình bày ở mức thiết kế và định hướng, làm cơ sở mở rộng thành đồ án tốt nghiệp.

4. Phương pháp nghiên cứu

- **Phương pháp nghiên cứu tài liệu:** đọc các đặc tả chuẩn (RFC 6749, RFC 7636, OIDC Core), tài liệu Keycloak và tài liệu Kubernetes.
- **Phương pháp phân tích – thiết kế:** phân tích yêu cầu, mô hình hóa kiến trúc (mô hình C4), vẽ biểu đồ tuần tự cho luồng xác thực.
- **Phương pháp thực nghiệm:** triển khai hệ thống thực tế, cấu hình, kiểm thử chức năng và đo lường.
- **Phương pháp đánh giá:** đánh giá theo các tiêu chí chức năng, hiệu năng và bảo mật (threat model STRIDE).

5. Ý nghĩa của chuyên đề

Về mặt khoa học, chuyên đề hệ thống hóa kiến thức về định danh tập trung và vận hành dịch vụ trọng yếu. Về mặt thực tiễn, kết quả của chuyên đề có thể được áp dụng để xây dựng hệ thống SSO nội bộ cho tổ chức, giảm chi phí quản trị và tăng cường bảo mật, đồng thời là nền tảng để phát triển thành đồ án tốt nghiệp ở mức độ hoàn chỉnh hơn.

6. Đóng góp của chuyên đề

Chuyên đề có một số đóng góp chính: (1) hệ thống hóa cơ sở lý thuyết và so sánh các giải pháp định danh phổ biến; (2) thiết kế kiến trúc ba tầng kết hợp dịch vụ định danh với quy trình tự động hóa; (3) triển khai thực nghiệm Keycloak cùng cấu hình định danh như mã nguồn và ứng dụng demo SSO; và (4) xây dựng khung quy trình CI/CD, GitOps, giám sát cùng phân tích bảo mật theo STRIDE.

7. Kế hoạch thực hiện

Bảng 2 trình bày kế hoạch thực hiện chuyên đề theo giai đoạn.

Bảng 2 - Kế hoạch thực hiện chuyên đề theo giai đoạn

Giai đoạn	Nội dung	Thời gian
1	Nghiên cứu lý thuyết và khảo sát công nghệ	Tuần 1–2
2	Phân tích yêu cầu và thiết kế kiến trúc	Tuần 3–4
3	Triển khai dịch vụ định danh và ứng dụng demo	Tuần 5–7
4	Tự động hóa và quản lý cấu hình như mã nguồn	Tuần 8–10
5	Đánh giá và hoàn thiện báo cáo	Tuần 11–14

8. Bố cục báo cáo

Ngoài phần Đặt vấn đề và Kết luận, báo cáo được tổ chức thành bốn chương:

- **Chương 1** trình bày cơ sở lý thuyết về xác thực tập trung, giao thức OAuth 2.0/OIDC, Keycloak và triển khai tự động.
- **Chương 2** phân tích yêu cầu và thiết kế kiến trúc hệ thống.
- **Chương 3** trình bày quá trình triển khai và cấu hình hệ thống.
- **Chương 4** đánh giá kết quả và thảo luận.

Chương 1

TỔNG QUAN VỀ XÁC THỰC TẬP TRUNG VÀ TRIỂN KHAI TỰ ĐỘNG

1.1 Xác thực, định danh và quản lý truy cập

1.1.1 Khái niệm xác thực và ủy quyền

Xác thực (Authentication) là quá trình kiểm tra danh tính của một chủ thể (người dùng hoặc dịch vụ) nhằm trả lời câu hỏi “bạn là ai?”. Ba yếu tố xác thực phổ biến gồm:

- **Yếu tố kiến thức** (something you know): mật khẩu, mã PIN.
- **Yếu tố sở hữu** (something you have): điện thoại, token, khóa bảo mật.
- **Yếu tố sinh trắc** (something you are): vân tay, khuôn mặt.

Khi kết hợp từ hai yếu tố trở lên, ta có xác thực đa yếu tố (MFA). Khác với xác thực, ủy quyền (Authorization) trả lời câu hỏi “bạn được phép làm gì?” và thường được mô hình hóa bằng vai trò (RBAC) hoặc chính sách chi tiết (ABAC). Xác thực và ủy quyền cùng thuộc lĩnh vực quản lý định danh và truy cập (IAM).

1.1.2 Đăng nhập một lần và định danh liên kết

Đăng nhập một lần (SSO) cho phép người dùng xác thực một lần và truy cập nhiều ứng dụng mà không phải đăng nhập lại. Trong mô hình định danh liên kết (Federated Identity), một nhà cung cấp định danh (IdP) chịu trách nhiệm xác thực, còn các ứng dụng – gọi là bên tin cậy (Relying Party – RP) – tin tưởng vào kết quả xác thực của IdP thông qua các giao thức chuẩn như OAuth 2.0, OpenID Connect hoặc SAML. Mô hình này cho phép tách biệt trách nhiệm xác thực khỏi logic nghiệp vụ của từng ứng dụng.

1.1.3 Các mối đe dọa phổ biến đối với hệ thống xác thực

Do nắm giữ “chìa khóa” truy cập vào mọi ứng dụng, hệ thống xác thực tập trung là mục tiêu hấp dẫn của kẻ tấn công. Một số mối đe dọa điển hình gồm:

- *Phishing*: tạo trang đăng nhập giả mạo để lừa người dùng nhập tài khoản và mật khẩu.
- *Brute force* và *credential stuffing*: thử liên tục nhiều mật khẩu, hoặc dùng thông tin đăng nhập rò rỉ từ các vụ vi phạm dữ liệu khác.

- *Session hijacking*: đánh cắp phiên đã xác thực (cookie, token) để mạo danh người dùng mà không cần mật khẩu.
- *Man-in-the-middle*: chặn bắt thông tin trên đường truyền nếu không được mã hóa.
- *Token replay*: đánh cắp và sử dụng lại token còn hiệu lực.
- *Privilege escalation*: lợi dụng cấu hình sai để nâng cao quyền.

Các biện pháp giảm thiểu tương ứng bao gồm: bắt buộc HTTPS/TLS, xác thực đa yếu tố, giới hạn số lần đăng nhập (rate limiting), đặt thời hạn token ngắn, sử dụng PKCE, ghi log kiểm toán và áp dụng nguyên tắc đặc quyền tối thiểu. Trong phạm vi chuyên đề, các biện pháp này được thể hiện xuyên suốt ở phần thiết kế và triển khai.

1.1.4 Lưu trữ thông tin xác thực an toàn

Một sai lầm nghiêm trọng là lưu mật khẩu dưới dạng thô (plaintext). Thay vào đó, mật khẩu phải được *băm* (hash) kèm một giá trị ngẫu nhiên gọi là *muối* (salt) bằng các thuật toán chuyên dụng dành cho mật khẩu như bcrypt, scrypt hoặc Argon2. Các thuật toán này được thiết kế để chậm một cách có chủ đích, khiến việc dò ngược trở nên tốn kém.

Keycloak hỗ trợ nhiều thuật toán băm và cho phép cấu hình chính sách mật khẩu như độ dài tối thiểu, yêu cầu ký tự, thời hạn và số lần đăng nhập sai. Việc sử dụng một hệ thống đã được kiểm chứng thay vì tự cài đặt cơ chế mã hóa là nguyên tắc quan trọng nhằm tránh các lỗ hổng phát sinh.

1.2 Giao thức OAuth 2.0 và OpenID Connect

1.2.1 OAuth 2.0

OAuth 2.0 [1] là một khung ủy quyền cho phép ứng dụng bên thứ ba truy cập tài nguyên có kiểm soát thay mặt người dùng mà không cần biết mật khẩu của họ. Các vai trò chính gồm: *Resource Owner* (người dùng), *Client* (ứng dụng), *Authorization Server* (máy chủ ủy quyền) và *Resource Server*. OAuth 2.0 định nghĩa nhiều luồng cấp quyền (*grant types*), trong đó luồng *Authorization Code* là luồng phù hợp nhất cho ứng dụng web.

1.2.2 OpenID Connect

OpenID Connect (OIDC) [2] được xây dựng trên nền OAuth 2.0, bổ sung lớp định danh thông qua *ID Token* – một JSON Web Token (JWT) [3] chứa các thông tin về người dùng. OIDC chuẩn hóa các điểm cuối (endpoint) như: /authorize, /token,

/userinfo và /.well-known/openid-configuration, đồng thời định nghĩa các *scope* như openid, profile, email.

1.2.3 Luồng Authorization Code và PKCE

Luồng *Authorization Code* hoạt động theo các bước tổng quát sau:

1. Người dùng truy cập ứng dụng (RP) và được chuyển hướng tới IdP.
2. IdP xác thực người dùng, sau đó chuyển hướng trở lại RP kèm theo một *authorization code* ngắn hạn.
3. RP trao đổi *authorization code* này với IdP để lấy *access token* và *ID Token*.
4. RP sử dụng token để truy cập tài nguyên hoặc xác lập phiên cho người dùng.

Để tăng cường bảo mật cho các ứng dụng công khai, phần mở rộng PKCE [4] (Proof Key for Code Exchange) được sử dụng: RP sinh một cặp giá trị *code_verifier* và *code_challenge*, gửi *code_challenge* khi bắt đầu luồng và gửi *code_verifier* khi đổi token, nhằm chống lại tấn công đánh chặn *authorization code*.

1.2.4 Cấu trúc JWT và xác thực token

JSON Web Token gồm ba phần được mã hóa Base64URL và ngăn cách bởi dấu chấm: *header* (thuật toán và loại token), *payload* (các khai báo – claims) và *signature* (chữ ký số). Một số claim chuẩn quan trọng gồm: *iss* (nhà phát hành), *sub* (chủ thể), *aud* (đối tượng nhận), *exp* (thời điểm hết hạn), *iat* (thời điểm phát hành), *nbf* (thời điểm bắt đầu hiệu lực) và *jti* (định danh token).

Khi nhận token, ứng dụng cần xác thực chữ ký bằng khóa công khai lấy từ điểm cuối JWKS của IdP, đồng thời kiểm tra các claim về nhà phát hành, đối tượng và thời hạn. Việc kiểm tra đầy đủ các bước này giúp chống lại token giả mạo hoặc token đã hết hiệu lực.

1.2.5 Các loại token và vòng đời

Trong OIDC/OAuth 2.0 tồn tại ba loại token chính: *access token* (dùng để truy cập tài nguyên, thời hạn ngắn), *refresh token* (dùng để xin *access token* mới khi hết hạn, thời hạn dài hơn) và *ID Token* (mang thông tin định danh người dùng). Việc đặt thời hạn ngắn cho *access token* giúp giảm thiệt hại nếu token bị đánh cắp, trong khi cơ chế *refresh token* giúp duy trì trải nghiệm người dùng mà không phải đăng nhập lại.

1.2.6 So sánh các luồng xác thực OIDC

Bảng 1.1 tóm tắt ưu nhược điểm của các luồng phổ biến và trường hợp sử dụng phù hợp.

Bảng 1.1 - So sánh các luồng xác thực OIDC

Luồng	Ưu điểm	Hạn chế / phù hợp
Authorization Code + PKCE	An toàn nhất, code trao đổi ở kênh sau	Ứng dụng web và di động
Implicit	Đơn giản, ít bước	Không an toàn; đã bị khuyến nghị loại bỏ
Hybrid	Kết hợp code và token	Phức tạp; ít dùng
Client Credentials	Không cần người dùng	Giao tiếp giữa các dịch vụ

Trên cơ sở so sánh, chuyên đề lựa chọn luồng *Authorization Code* kèm PKCE cho ứng dụng demo, đây là khuyến nghị hiện hành cho ứng dụng web.

1.3 Giải pháp xác thực tập trung: Keycloak

1.3.1 Giới thiệu Keycloak

Keycloak [5] là một giải pháp mã nguồn mở về quản lý định danh và truy cập, hỗ trợ sẵn SSO, OIDC, OAuth 2.0, SAML 2.0, xác thực đa yếu tố và liên kết với các nguồn định danh bên ngoài (LDAP, Active Directory, mạng xã hội). Keycloak được viết trên nền Quarkus, có khả năng mở rộng và được sử dụng rộng rãi trong cả môi trường doanh nghiệp lẫn học thuật. Đây là lý do Keycloak được lựa chọn làm nền tảng IdP cho chuyên đề.

1.3.2 Các khái niệm chính

- **Realm:** một không gian quản lý định danh độc lập, chứa người dùng, vai trò và các client.
- **Client:** một ứng dụng hoặc dịch vụ được phép sử dụng Keycloak để xác thực.
- **Role:** vai trò dùng cho ủy quyền (RBAC).
- **User:** tài khoản người dùng thuộc một realm.
- **Admin REST API:** giao diện lập trình cho phép tự động hóa cấu hình và quản trị Keycloak.

1.3.3 So sánh với một số giải pháp khác

Bảng 1.2 so sánh Keycloak với một số giải pháp định danh phổ biến.

1.3.4 Kiến trúc nội bộ của Keycloak

Keycloak được xây dựng trên nền tảng Quarkus và tổ chức thành các lớp: lớp điểm cuối REST xử lý yêu cầu giao thức, lớp dịch vụ chứa logic nghiệp vụ, lớp lưu trữ

Bảng 1.2 - So sánh Keycloak với một số giải pháp định danh

Tiêu chí	Keycloak	Okta/Auth0	Ory Hydra	Authentik
Mã nguồn mở	Có (Apache)	Không	Có	Có
Tự vận hành	Có	Hạn chế	Có	Có
OIDC/OAuth2/SAML	Có	Có	OAuth2/OIDC	Có
Giao diện quản trị	Có	Có	Tối giản	Có
Độ phổ biến / tài liệu	Cao	Cao	Trung bình	Trung bình
Chi phí	Miễn phí	Trả phí	Miễn phí	Miễn phí

(JPA) làm việc với cơ sở dữ liệu, và lớp bộ nhớ đệm Infinispan dùng cho phiên đăng nhập cùng dữ liệu realm. Kiến trúc này cho phép Keycloak chạy nhiều bản sao trong mô hình cụm để đảm bảo tính sẵn sàng cao.

Keycloak cung cấp nhiều điểm mở rộng (SPI) cho xác thực, ánh xạ giao thức và lưu trữ, cùng Admin REST API cho phép tự động hóa toàn bộ thao tác quản trị. Nó cũng hỗ trợ xuất/nhập cấu hình realm dưới dạng JSON và cung cấp các điểm cuối về sức khỏe (health) và chỉ số (metrics) phục vụ giám sát. Những đặc điểm này là nền tảng để quản lý cấu hình như mã nguồn và triển khai tự động.

1.3.5 Các chuẩn và giao thức liên quan

Ngoài OAuth 2.0 và OIDC, trong lĩnh vực định danh còn tồn tại nhiều chuẩn khác:

- *SAML 2.0*: giao thức định danh liên kết dựa trên XML, phổ biến trong môi trường doanh nghiệp.
- *LDAP*: giao thức truy cập thư mục, thường được dùng làm nguồn người dùng.
- *Kerberos*: giao thức xác thực phổ biến trong mạng nội bộ.
- *WebAuthn*: chuẩn xác thực không mật khẩu dựa trên khóa công khai.

Keycloak hỗ trợ liên kết với LDAP/Active Directory và SAML, nhờ đó có thể tích hợp vào hạ tầng định danh sẵn có của tổ chức mà không cần thay thế toàn bộ.

1.4 Ảo hóa container và Kubernetes

1.4.1 Container hóa với Docker

Docker đóng gói ứng dụng cùng các phụ thuộc vào một *container* – một đơn vị chạy nhẹ, nhất quán giữa các môi trường. Image của Keycloak, PostgreSQL và ứng dụng demo đều có thể được phân phối dưới dạng image, giúp việc triển khai trở nên tái lập được.

1.4.2 Kubernetes và k3s

Kubernetes [6] là nền tảng điều phối container, cung cấp khả năng tự phục hồi, mở rộng và quản lý vòng đời dịch vụ. Trong phạm vi chuyên đề, bản phân phối nhẹ k3s được sử dụng để dựng cụm Kubernetes ngay trên máy cá nhân hoặc trên máy ảo, phù hợp với điều kiện thực hành hạn chế về hạ tầng.

1.4.3 Helm

Helm là trình quản lý gói cho Kubernetes, cho phép đóng gói các manifest thành *chart* có thể tham số hóa và tái sử dụng. Keycloak có chart chính thức, thuận lợi cho việc triển khai tự động.

1.4.4 Các đối tượng cốt lõi của Kubernetes

Kubernetes quản lý ứng dụng thông qua các đối tượng khai báo. Một số đối tượng quan trọng:

- *Pod*: đơn vị nhỏ nhất, chứa một hoặc nhiều container cùng chạy.
- *Deployment*: quản lý việc tạo và cập nhật các bản sao của Pod, hỗ trợ cuộn chiều và tự phục hồi.
- *Service*: cung cấp địa chỉ ổn định và cân bằng tải cho một nhóm Pod.
- *Ingress*: định tuyến lưu lượng HTTP/HTTPS từ bên ngoài vào các Service.
- *ConfigMap* và *Secret*: tách cấu hình và thông tin nhạy cảm khỏi image.
- *StatefulSet* và *PersistentVolume*: quản lý các ứng dụng có trạng thái như cơ sở dữ liệu.
- *HorizontalPodAutoscaler*: tự động mở rộng số bản sao theo tải.

Việc hiểu rõ các đối tượng này là điều kiện cần để xây dựng manifest cho dịch vụ định danh và ứng dụng demo.

1.4.5 Container hoạt động như thế nào

Container là một tiến trình được cô lập nhờ các tính năng của nhân Linux: *namespace* (cô lập không gian tên, mạng, tiến trình) và *cgroup* (giới hạn tài nguyên CPU, bộ nhớ). Khác với máy ảo, container chia sẻ nhân với máy chủ nên nhẹ và khởi động nhanh. Image được tổ chức thành nhiều lớp (layer) xếp chồng, giúp tái sử dụng và phân phối hiệu quả.

1.4.6 Kiến trúc của Kubernetes

Một cụm Kubernetes gồm hai phần: *control plane* (máy chủ điều khiển) và các *node* (máy công tác). Control plane chứa API server (điểm vào duy nhất), etcd (lưu trạng thái), scheduler (phân bổ Pod) và controller manager (duy trì trạng thái mong muốn). Trên mỗi node, kubelet quản lý container và kube-proxy xử lý mạng. k3s là bản phân phối gọn nhẹ, đóng gói các thành phần này vào một tiến trình duy nhất, phù hợp cho môi trường học tập và thiết bị hạn chế.

1.5 Triển khai tự động

1.5.1 Tích hợp và triển khai liên tục (CI/CD)

CI/CD tự động hóa việc kiểm thử, đóng gói và phát hành phần mềm. Mỗi thay đổi được đẩy lên kho mã nguồn sẽ kích hoạt một *pipeline* thực hiện: kiểm tra mã (lint), chạy kiểm thử, xây dựng image, quét lỗ hổng và phát hành. Điều này giúp giảm sai sót do thao tác thủ công.

1.5.2 Hạ tầng dưới dạng mã (IaC)

IaC (Infrastructure as Code) mô tả hạ tầng bằng các tệp cấu hình (ví dụ Terraform, Ansible) thay vì thao tác thủ công, cho phép tái lập môi trường và quản lý hạ tầng bằng quy trình phiên bản hóa.

1.5.3 GitOps và ArgoCD

GitOps lấy kho Git làm nguồn chân lý duy nhất cho trạng thái mong muốn của hệ thống. ArgoCD [7] liên tục so sánh trạng thái thực tế trên cụm Kubernetes với trạng thái trong Git và tự động đồng bộ, giúp việc triển khai trở nên minh bạch, có thể truy vết và dễ dàng khôi phục.

1.5.4 Quản lý cấu hình như mã nguồn

Đối với Keycloak, cấu hình định danh (realm, client, role, chính sách) cũng có thể được quản lý như mã nguồn thông qua các công cụ như keycloak-config-cli [8] hoặc Keycloak Terraform Provider [9]. Đây là cầu nối giữa dịch vụ định danh và quy trình triển khai tự động, là trọng tâm của chuyên đề.

1.5.5 Lợi ích và thách thức của triển khai tự động

Triển khai tự động mang lại nhiều lợi ích: khả năng tái lập môi trường, tính nhất quán giữa các lần triển khai, giảm thời gian khắc phục sự cố và tăng tốc độ phát hành. Mọi thay đổi đều có thể truy vết qua lịch sử Git, giúp việc kiểm toán và khôi phục trở nên dễ dàng.

Tuy nhiên, hướng tiếp cận này cũng đặt ra thách thức: đòi hỏi kỹ năng và thời

gian đầu tư ban đầu, việc quản lý bí mật (khóa, mật khẩu) trở nên phức tạp hơn, bản thân pipeline cũng cần được kiểm thử, và lỗi cấu hình có thể lan rộng nhanh. Do đó, nguyên tắc “tự động hóa có kiểm soát” – kết hợp kiểm thử tự động và khả năng quay lui – là cần thiết.

1.5.6 Ví dụ một quy trình CI/CD điển hình

Một quy trình CI/CD hoàn chỉnh thường trải qua các giai đoạn sau: (1) lấy mã nguồn và kiểm tra định dạng (lint); (2) chạy kiểm thử đơn vị; (3) xây dựng image; (4) quét lỗ hổng bảo mật; (5) đẩy image lên registry; (6) cập nhật manifest và để GitOps đồng bộ; (7) kiểm tra sau triển khai (smoke test). Mỗi giai đoạn có thể chặn quy trình nếu thất bại, đảm bảo chỉ những thay đổi đạt chất lượng mới được phát hành.

1.6 Kết chương

Chương này đã trình bày cơ sở lý thuyết về xác thực tập trung, giao thức OAuth 2.0/OIDC, nền tảng Keycloak và các phương pháp triển khai tự động. Trên nền tảng đó, chương tiếp theo sẽ phân tích yêu cầu và thiết kế kiến trúc cho hệ thống cụ thể của chuyên đề.

Chương 2

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu

2.1.1 Yêu cầu chức năng

Bảng 2.1 - Danh sách yêu cầu chức năng

Mã	Yêu cầu chức năng
FN1	Hệ thống cung cấp dịch vụ định danh tập trung dựa trên Keycloak.
FN2	Người dùng đăng nhập một lần (SSO) vào ứng dụng demo qua giao thức OIDC.
FN3	Quản trị viên tạo và quản lý realm, client, role, user dưới dạng mã nguồn.
FN4	Hệ thống hỗ trợ xác thực đa yếu tố (TOTP).
FN5	Cấu hình định danh được áp dụng tự động qua quy trình CI/CD.
FN6	Ứng dụng demo hiển thị thông tin người dùng nhận từ token.

2.1.2 Yêu cầu phi chức năng

Bảng 2.2 - Danh sách yêu cầu phi chức năng

Mã	Yêu cầu phi chức năng
NF1	Bảo mật: sử dụng HTTPS/TLS, luồng Authorization Code + PKCE, không lưu mật khẩu dạng thô.
NF2	Tự động hóa: có thể tái lập toàn bộ hệ thống từ mã nguồn.
NF3	Khả năng bảo trì: cấu hình tách khỏi image, quản lý bằng Git.
NF4	Hiệu năng: đáp ứng đăng nhập với độ trễ chấp nhận được ở quy mô thử nghiệm.
NF5	Khả năng mở rộng: kiến trúc sẵn sàng chạy nhiều bản sao và tự phục hồi.

2.1.3 Tác nhân và ca sử dụng

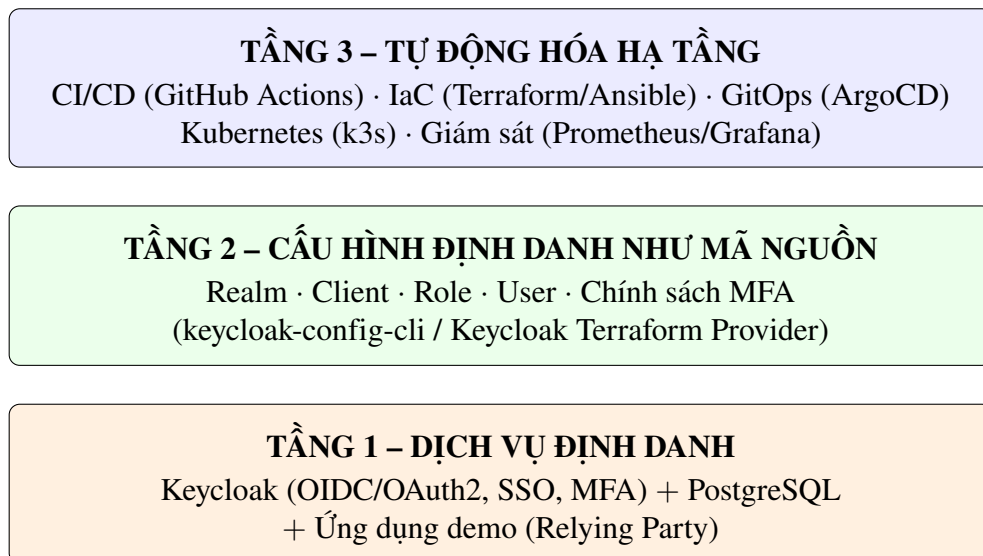
Bảng 2.3 liệt kê các tác nhân và ca sử dụng chính của hệ thống.

Bảng 2.3 - Tác nhân và ca sử dụng chính

Tác nhân	Ca sử dụng	Mô tả
Người dùng	Đăng nhập SSO	Xác thực qua Keycloak và truy cập ứng dụng demo
Người dùng	Bật MFA	Thiết lập và sử dụng mã TOTP
Quản trị viên	Quản lý realm/client	Tạo, sửa cấu hình định danh dưới dạng mã nguồn
Quản trị viên	Triển khai	Áp dụng cấu hình qua pipeline CI/CD
Hệ thống CI	Kiểm thử và build	Chạy kiểm thử, build và đẩy image
ArgoCD	Đồng bộ	Đồng bộ trạng thái mong muốn từ Git lên cụm

2.2 Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc ba tầng như minh họa trên Hình 2.1. Tầng dưới cùng cung cấp dịch vụ định danh; tầng giữa chịu trách nhiệm quản lý cấu hình định danh như mã nguồn; tầng trên cùng đảm nhiệm tự động hóa hạ tầng và vận hành.



Hình 2.1 - Kiến trúc tổng thể ba tầng của hệ thống

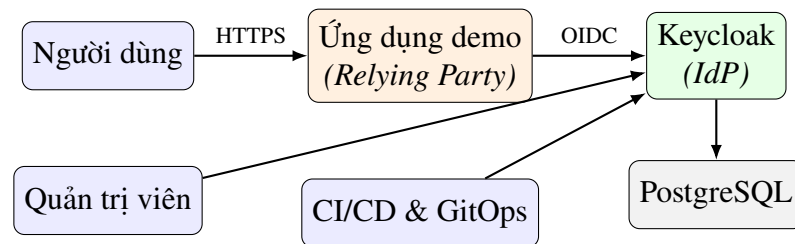
Việc tách thành ba tầng giúp phân định rõ trách nhiệm: tầng dịch vụ tập trung vào logic định danh, tầng cấu hình đảm bảo tính nhất quán và có thể truy vết của cấu hình, còn tầng tự động hóa chịu trách nhiệm triển khai và vận hành.

2.2.1 Mô hình kiến trúc C4

Để mô tả kiến trúc một cách có hệ thống, chuyên đề sử dụng mô hình C4 ở hai mức: sơ đồ ngữ cảnh (context) và sơ đồ container. Ở mức ngữ cảnh, hệ thống gồm ba nhóm tác nhân: người dùng cuối, quản trị viên và các hệ thống tự động (CI/CD,

GitOps). Người dùng tương tác với ứng dụng demo; ứng dụng demo tin cậy Keycloak để xác thực; quản trị viên và hệ thống tự động tương tác với Keycloak và hạ tầng qua API.

Ở mức container (Hình 2.2), hệ thống gồm các thành phần: ứng dụng demo đóng vai trò Relying Party, Keycloak (IdP), PostgreSQL lưu trữ, cơ chế cấu hình như mã nguồn, cụm Kubernetes (k3s), ArgoCD, pipeline CI và hệ thống giám sát.



Hình 2.2 - Sơ đồ container của hệ thống (mức C4)

2.3 Thiết kế luồng xác thực OIDC

Luồng xác thực trong hệ thống tuân theo quy trình Authorization Code kèm PKCE:

1. Người dùng truy cập ứng dụng demo và chọn “Đăng nhập”.
2. Ứng dụng chuyển hướng người dùng tới Keycloak kèm `code_challenge`.
3. Keycloak xác thực người dùng (mật khẩu và, nếu được bật, mã TOTP).
4. Keycloak chuyển hướng trở lại ứng dụng kèm `authorization code`.
5. Ứng dụng gửi `authorization code` và `code_verifier` tới endpoint `/token` để đổi lấy ID Token và Access Token.
6. Ứng dụng xác thực token và thiết lập phiên cho người dùng.

Bảng 2.4 mô tả chi tiết từng bước của luồng, bao gồm tác nhân và dữ liệu trao đổi.

2.4 Thiết kế quản lý cấu hình định danh như mã nguồn

Toàn bộ cấu hình định danh được mô tả trong kho Git, gồm:

- Định nghĩa realm và các thuộc tính bảo mật (thời hạn token, chính sách mật khẩu).
- Định nghĩa client cho ứng dụng demo (loại confidential/public, redirect URI, phạm vi scope).

Bảng 2.4 - Các bước của luồng Authorization Code kèm PKCE

Bước	Tác nhân	Hành động và dữ liệu
1	Người dùng Ứng dụng	→ Chọn đăng nhập.
2	Ứng dụng Keycloak	→ Chuyển hướng tới /authorize kèm code_challenge, state.
3	Keycloak Người dùng	→ Hiển thị trang đăng nhập và xác thực (mật khẩu, TOTP).
4	Keycloak Ứng dụng	→ Chuyển hướng về redirect_uri kèm authorization code.
5	Ứng dụng Keycloak	→ Gửi code và code_verifier tới /token.
6	Keycloak Ứng dụng	→ Trả về ID Token, access token, refresh token.
7	Ứng dụng Người dùng	→ Xác thực token và thiết lập phiên.

- Định nghĩa role và ánh xạ vai trò.
- Cấu hình MFA (TOTP).

Khi cấu hình thay đổi trong Git, một pipeline sẽ áp dụng thay đổi đó vào Keycloak, đảm bảo môi trường luôn nhất quán và không phụ thuộc thao tác thủ công trên giao diện quản trị.

2.5 Thiết kế quy trình triển khai tự động

Quy trình triển khai được thiết kế như sau:

1. Lập trình viên đẩy thay đổi lên kho Git.
2. Hệ thống CI (GitHub Actions) chạy kiểm thử, xây dựng image ứng dụng và đẩy lên registry.
3. Các manifest Kubernetes được cập nhật trong kho Git (hoặc kho riêng cho cấu hình).
4. ArgoCD phát hiện thay đổi và đồng bộ trạng thái mong muốn lên cụm k3s.
5. Hệ thống giám sát (Prometheus/Grafana) theo dõi trạng thái dịch vụ.

Cơ chế này cho phép triển khai theo kiểu cuộn chiếu (rolling update) và tự động khôi phục khi có sự cố, hạn chế gián đoạn dịch vụ.

2.6 Thiết kế cơ sở dữ liệu và trạng thái phiên

Keycloak lưu trữ dữ liệu realm, người dùng, client và phiên đăng nhập trong cơ sở dữ liệu quan hệ (PostgreSQL). Bên cạnh đó, phiên và dữ liệu realm được đệm trong bộ nhớ Infinispan để tăng tốc truy cập. Bảng 2.5 tóm tắt các nhóm dữ liệu chính.

Bảng 2.5 - Các nhóm dữ liệu chính trong hệ thống

Nhóm dữ liệu	Lưu trữ	Mô tả
Cấu hình realm/client	PostgreSQL	Định nghĩa realm, client, role
Tài khoản người dùng	PostgreSQL	Thông tin người dùng và mật khẩu đã băm
Phiên đăng nhập	Infinispan cookie	+ Trạng thái SSO của người dùng
Token	Không (stateless)	lưu Được sinh và xác thực theo chữ ký

2.7 Thiết kế tính sẵn sàng cao

Để đảm bảo dịch vụ định danh luôn sẵn sàng, kiến trúc được thiết kế với nhiều bản sao Keycloak đặt sau bộ cân bằng tải, cơ sở dữ liệu có sao chép và phiên được đệm phân tán. Các điểm cuối kiểm tra sức khỏe (liveness/readiness) giúp Kubernetes tự phát hiện và thay thế Pod lỗi, trong khi cơ chế cuộn chiếu đảm bảo cập nhật không gián đoạn. Bộ tự động mở rộng (HPA) giúp tăng số bản sao khi tải tăng.

2.8 Thiết kế bảo mật

Trên cơ sở các mối đe dọa đã phân tích ở Chương 1, Bảng 2.6 ánh xạ từng nhóm mối đe dọa với biện pháp thiết kế tương ứng.

Bảng 2.6 - Ánh xạ mối đe dọa và biện pháp thiết kế

Mối đe dọa	Biện pháp thiết kế
Nghe lén / MITM	Bắt buộc TLS cho mọi kết nối
Đánh cắp phiên/token	Thời hạn token ngắn, PKCE, cookie an toàn
Dò mật khẩu	Giới hạn tần suất, chính sách mật khẩu, MFA
Truy cập trái phép	RBAC, đặc quyền tối thiểu, NetworkPolicy
Lộ bí mật	Quản lý bí mật bằng Secret/Vault, không nhúng vào mã nguồn
Sửa đổi cấu hình	Quản lý cấu hình qua Git, kiểm soát phiên bản

2.9 Thiết kế cấu trúc kho mã nguồn

Kho mã nguồn được tổ chức thành hai phần: mã nguồn ứng dụng (kèm Dockerfile) và mã nguồn vận hành (manifest Kubernetes, cấu hình realm, pipeline CI, khai báo GitOps). Việc tách nhánh theo môi trường (dev, staging, prod) và sử dụng pull request giúp kiểm soát thay đổi trước khi áp dụng. Bảng 2.7 mô tả cấu trúc thư mục dự kiến.

Bảng 2.7 - Cấu trúc thư mục kho mã nguồn dự án

Thư mục	Nội dung
app/ deploy/ k8s/	Mã nguồn ứng dụng demo và Dockerfile Tập docker-compose và cấu hình realm Manifest Kubernetes (Deployment, Service, Ingress, HPA)
gitops/ .github/ docs/	Khai báo ArgoCD và các overlay theo môi trường Pipeline CI/CD Tài liệu, sơ đồ thiết kế

2.10 Thiết kế quản lý bí mật

Thông tin nhạy cảm như mật khẩu cơ sở dữ liệu, khóa ký token và mật khẩu quản trị tuyệt đối không được lưu trong mã nguồn. Ở môi trường thử nghiệm, chúng được đưa vào qua biến môi trường hoặc tệp không được phiên bản hóa; ở môi trường triển khai, chúng được quản lý bằng Kubernetes Secret, Sealed Secrets hoặc HashiCorp Vault. Nguyên tắc chung là tách bí mật khỏi mã và chỉ cấp cho thành phần cần dùng.

2.11 Thiết kế thành phần hệ thống

Hệ thống gồm các thành phần chính: (1) ứng dụng demo – giao diện người dùng và xử lý luồng OIDC; (2) Keycloak – dịch vụ định danh trung tâm; (3) PostgreSQL – lưu trữ dữ liệu định danh; (4) công cụ cấu hình như mã nguồn; (5) pipeline CI; (6) ArgoCD; và (7) hệ thống giám sát. Mỗi thành phần có trách nhiệm riêng và giao tiếp qua các giao diện xác định, giúp hệ thống dễ thay thế và mở rộng.

2.12 Kết chương

Chương này đã phân tích yêu cầu chức năng, phi chức năng và thiết kế kiến trúc ba tầng của hệ thống, cùng các luồng xác thực và quy trình triển khai tự động. Chương tiếp theo trình bày quá trình triển khai thực tế dựa trên thiết kế này.

Chương 3

TRIỂN KHAI VÀ CẤU HÌNH HỆ THỐNG

3.1 Môi trường và công cụ

Hệ thống được triển khai thử nghiệm trên máy cá nhân chạy hệ điều hành Linux, với các công cụ chính được liệt kê trong Bảng 3.1.

Bảng 3.1 - Các công cụ và công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Nhà cung cấp định danh	Keycloak 26	Cung cấp OIDC/OAuth2, SSO, MFA
Cơ sở dữ liệu	PostgreSQL 16	Lưu trữ dữ liệu người dùng, realm
Container	Docker	Đóng gói và chạy dịch vụ
Orchestration	k3s	Cụm Kubernetes nhẹ
Quản lý gói K8s	Helm	Triển khai Keycloak theo chart
CI/CD	GitHub Actions	Tự động hóa kiểm thử và build
GitOps	ArgoCD	Đồng bộ trạng thái từ Git
Giám sát	Prometheus/Grafana	Thu thập và trực quan hóa chỉ số

3.2 Triển khai Keycloak và PostgreSQL bằng Docker Compose

Ở giai đoạn đầu, Keycloak và PostgreSQL được triển khai cục bộ bằng Docker Compose nhằm làm quen và kiểm thử luồng xác thực trước khi đưa lên Kubernetes. Mã nguồn cấu hình tham khảo được trình bày trong Đoạn mã 3.1.

```
1 services:
2   postgres:
3     image: postgres:16
4     environment:
5       POSTGRES_DB: keycloak
6       POSTGRES_USER: keycloak
7       POSTGRES_PASSWORD: ${DB_PASSWORD}
8     volumes:
9       - pgdata:/var/lib/postgresql/data
10    healthcheck:
11      test: ["CMD-SHELL", "pg_isready -U keycloak"]
12      interval: 10s
13      retries: 5
14
```

```

15 keycloak:
16   image: quay.io/keycloak/keycloak:26.0
17   command: start-dev --import-realm
18   environment:
19     KC_DB: postgres
20     KC_DB_URL: jdbc:postgresql://postgres:5432/keycloak
21     KC_DB_USERNAME: keycloak
22     KC_DB_PASSWORD: ${DB_PASSWORD}
23     KC_HOSTNAME: localhost
24     KC_METRICS_ENABLED: "true"
25     KC_HEALTH_ENABLED: "true"
26     KEYCLOAK_ADMIN: admin
27     KEYCLOAK_ADMIN_PASSWORD: ${KC_ADMIN_PASSWORD}
28   ports:
29     - "8080:8080"
30   depends_on:
31     postgres:
32       condition: service_healthy
33
34 volumes:
35   pgdata:

```

Đoạn mã 3.1 - Cấu hình Docker Compose cho Keycloak và PostgreSQL

Việc tách cấu hình nhạy cảm (mật khẩu) ra biến môi trường giúp tránh đưa thông tin bí mật vào mã nguồn – một yêu cầu quan trọng của thiết kế an toàn.

3.2.1 Kiểm tra và xác thực dịch vụ

Sau khi khởi chạy, cần kiểm tra dịch vụ hoạt động đúng trước khi cấu hình ứng dụng. Đoạn mã 3.2 minh họa việc đọc tài liệu khám phá OIDC và xin token bằng luồng client credentials.

```

1 # Doc tai lieu discovery
2 curl -s http://localhost:8080/realms/demo/.well-known/openid-
   configuration | jq .
3
4 # Xin token bang client credentials
5 curl -s -X POST \
6   http://localhost:8080/realms/demo/protocol/openid-connect/token \
7   -d grant_type=client_credentials \
8   -d client_id=demo-app \
9   -d client_secret=$CLIENT_SECRET | jq .

```

Đoạn mã 3.2 - Kiểm tra điểm cuối OIDC của Keycloak

3.3 Cấu hình định danh như mã nguồn

Cấu hình realm, client và role được mô tả trong một tệp JSON và nạp tự động khi Keycloak khởi động (tham số `-import-realm`). Một phần nội dung được minh họa trong Đoạn mã 3.3.

```

1 {
2   "realm": "demo",
3   "enabled": true,
4   "sslRequired": "external",
5   "accessTokenLifespan": 300,
6   "otpPolicyType": "totp",
7   "clients": [
8     {
9       "clientId": "demo-app",
10      "protocol": "openid-connect",
11      "publicClient": false,
12      "standardFlowEnabled": true,
13      "redirectUri": ["http://localhost:3000/callback"]
14    }
15  ],
16  "roles": {
17    "realm": [
18      { "name": "user" },
19      { "name": "admin" }
20    ]
21  }
22 }
```

Đoạn mã 3.3 - Trích tệp cấu hình realm dưới dạng mã nguồn

Trong giai đoạn nâng cao, cấu hình này được quản lý bằng `keycloak-config-cli` [8] hoặc Keycloak Terraform Provider [9] để có thể áp dụng lặp lại (idempotent) và kiểm soát phiên bản chặt chẽ hơn.

3.3.1 Áp dụng cấu hình tự động với `keycloak-config-cli`

Để cấu hình được áp dụng lặp lại và tự động (idempotent), công cụ `keycloak-config-cli` được sử dụng. Công cụ này đọc các tệp JSON từ một thư mục và đồng bộ chúng lên Keycloak, phù hợp để tích hợp vào pipeline CI. Đoạn mã 3.4 minh họa cách gọi công

cụ.

```

1 docker run --rm \
2   -e KEYCLOAK_URL=http://keycloak:8080 \
3   -e KEYCLOAK_USER=admin \
4   -e KEYCLOAK_PASSWORD=$KC_ADMIN_PASSWORD \
5   -v $(pwd)/realm:/config \
6   quay.io/adorsys/keycloak-config-cli:latest \
7   --import.files.locations=/config/demo-realm.json

```

Đoạn mã 3.4 - Áp dụng cấu hình realm bằng keycloak-config-cli

Nhờ cách tiếp cận này, mọi thay đổi cấu hình đều đi qua Git, có thể xem lại lịch sử và khôi phục khi cần, thay vì thao tác thủ công trên giao diện quản trị.

3.4 Ứng dụng demo và đăng nhập SSO

Ứng dụng demo đóng vai trò Relying Party, sử dụng thư viện OIDC để thực hiện luồng Authorization Code kèm PKCE với Keycloak. Sau khi đăng nhập thành công, ứng dụng nhận ID Token, xác thực chữ ký token và hiển thị thông tin người dùng (họ tên, email, vai trò). Đây là kịch bản minh họa trực quan cho tính năng SSO của hệ thống.

3.4.1 Mã nguồn ứng dụng demo

Ứng dụng demo được viết bằng Node.js với thư viện openid-client, thực hiện luồng Authorization Code kèm PKCE. Đoạn mã 3.5 trình bày phần xử lý đăng nhập và nhận token.

```

1 const express = require('express');
2 const session = require('express-session');
3 const { Issuer, generators } = require('openid-client');
4
5 const app = express();
6 app.use(session({ secret: process.env.SESSION_SECRET,
7   resave: false, saveUninitialized: false }));
8
9 let client;
10 async function init() {
11   const issuer = await Issuer.discover('http://localhost:8080/realms/
12     demo');
13   client = new issuer.Client({
14     client_id: 'demo-app',
15     client_secret: process.env.CLIENT_SECRET,

```

```

15     redirect_uris: ['http://localhost:3000/callback'],
16     response_types: ['code']
17   });
18 }
19
20 app.get('/login', (req, res) => {
21   const code_verifier = generators.codeVerifier();
22   const code_challenge = generators.codeChallenge(code_verifier);
23   req.session.code_verifier = code_verifier;
24   const url = client.authorizationUrl({
25     scope: 'openid profile email',
26     code_challenge,
27     code_challenge_method: 'S256'
28   });
29   res.redirect(url);
30 });
31
32 app.get('/callback', async (req, res) => {
33   const params = client.callbackParams(req);
34   const tokenSet = await client.callback(
35     'http://localhost:3000/callback', params,
36     { code_verifier: req.session.code_verifier });
37   req.session.user = tokenSet.claims();
38   res.redirect('/');
39 });
40
41 init().then(() => app.listen(3000));

```

Đoạn mã 3.5 - Xử lý đăng nhập OIDC trong ứng dụng demo (Node.js)

3.5 Đóng gói và triển khai trên Kubernetes

Sau khi kiểm thử cục bộ, hệ thống được thiết kế để triển khai trên cụm k3s. Keycloak được triển khai thông qua Helm chart với các giá trị tùy biến (Đoạn mã 3.6), cho phép cấu hình số bản sao, tài nguyên và kết nối cơ sở dữ liệu.

```

1 replicas: 2
2 image:
3   repository: quay.io/keycloak/keycloak
4   tag: "26.0"
5 command:
6   - start

```

```

7   - --optimized
8   extraEnv: |
9     - name: KC_DB
10      value: postgres
11     - name: KC_HEALTH_ENABLED
12      value: "true"
13     - name: KC_METRICS_ENABLED
14      value: "true"
15   resources:
16     requests:
17       cpu: "250m"
18       memory: "512Mi"
19     limits:
20       cpu: "1"
21       memory: "1Gi"

```

Đoạn mã 3.6 - Trích tệp values.yaml cho Helm chart Keycloak

3.5.1 Manifest triển khai Keycloak

Đoạn mã 3.7 và 3.8 trình bày manifest Deployment và Service cho Keycloak, kèm các đầu dò sức khỏe để Kubernetes tự phát hiện Pod lỗi.

```

1   apiVersion: apps/v1
2   kind: Deployment
3   metadata:
4     name: keycloak
5     namespace: identity
6   spec:
7     replicas: 2
8     selector:
9       matchLabels:
10        app: keycloak
11   template:
12     metadata:
13       labels:
14        app: keycloak
15     spec:
16       containers:
17         - name: keycloak
18           image: quay.io/keycloak/keycloak:26.0
19           args: ["start", "--optimized"]
20           envFrom:

```

```

21         - secretRef:
22             name: keycloak-secret
23     ports:
24         - containerPort: 8080
25     readinessProbe:
26         httpGet: { path: /health/ready, port: 8080 }
27     livenessProbe:
28         httpGet: { path: /health/live, port: 8080 }

```

Đoạn mã 3.7 - Manifest Deployment cho Keycloak

```

1  apiVersion: v1
2  kind: Service
3  metadata:
4      name: keycloak
5      namespace: identity
6  spec:
7      selector:
8          app: keycloak
9      ports:
10         - port: 8080
11           targetPort: 8080
12  ---
13  apiVersion: autoscaling/v2
14  kind: HorizontalPodAutoscaler
15  metadata:
16      name: keycloak
17      namespace: identity
18  spec:
19      scaleTargetRef:
20         apiVersion: apps/v1
21         kind: Deployment
22         name: keycloak
23      minReplicas: 2
24      maxReplicas: 5
25      metrics:
26         - type: Resource
27           resource:
28             name: cpu
29           target:
30             type: Utilization
31             averageUtilization: 70

```

Đoạn mã 3.8 - Service, Ingress và HorizontalPodAutoscaler

3.6 Tự động hóa bằng CI/CD và GitOps

Quy trình CI/CD được xây dựng bằng GitHub Actions. Mỗi lần đẩy mã nguồn, hệ thống tự động kiểm thử, xây dựng image ứng dụng và đẩy lên registry (Đoạn mã 3.9).

```

1 name: ci
2 on:
3   push:
4     branches: [ main ]
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - name: Build image
11        run: docker build -t ghcr.io/${{ github.repository }}:${{ github.
12          sha }} .
13      - name: Scan image
14        run: trivy image ghcr.io/${{ github.repository }}:${{ github.sha
15          }}
16      - name: Push image
17        run: docker push ghcr.io/${{ github.repository }}:${{ github.sha
18          }}

```

Đoạn mã 3.9 - Trích quy trình CI bằng GitHub Actions

Về phía triển khai, ArgoCD [7] theo dõi kho Git chứa manifest và tự động đồng bộ lên cụm k3s (Đoạn mã 3.10). Nhờ vậy, trạng thái của hệ thống luôn phản ánh đúng cấu hình trong Git.

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: keycloak
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: https://github.com/<user>/<repo>.git
10    path: k8s/keycloak

```

```

11   targetRevision: main
12   destination:
13     server: https://kubernetes.default.svc
14     namespace: identity
15   syncPolicy:
16     automated:
17       prune: true
18       selfHeal: true

```

Đoạn mã 3.10 - Khai báo ứng dụng ArgoCD

3.7 Giám sát và vận hành

Để vận hành dịch vụ định danh một cách tin cậy, cần thu thập chỉ số và log. Keycloak hỗ trợ xuất chỉ số qua điểm cuối `/metrics` khi được bật. Cấu hình Prometheus thu thập các chỉ số này được minh họa trong Đoạn mã 3.11.

```

1 scrape_configs:
2   - job_name: keycloak
3     metrics_path: /metrics
4     static_configs:
5       - targets: ["keycloak.identity.svc:8080"]

```

Đoạn mã 3.11 - Cấu hình Prometheus thu thập chỉ số Keycloak

Trên Grafana, các bảng theo dõi chính gồm: số lần đăng nhập thành công/thất bại, tỉ lệ cấp token, độ trễ xử lý yêu cầu, mức sử dụng CPU/bộ nhớ của các Pod và số lượng phiên đang hoạt động. Việc giám sát giúp phát hiện sớm bất thường như tấn công dò mật khẩu hoặc quá tải dịch vụ.

3.8 Quản lý TLS với cert-manager

Để đảm bảo mọi kết nối đều được mã hóa, chứng chỉ TLS được cấp và gia hạn tự động thông qua cert-manager kết hợp với Let's Encrypt. Đoạn mã 3.12 trình bày khai báo một ClusterIssuer.

```

1 apiVersion: cert-manager.io/v1
2 kind: ClusterIssuer
3 metadata:
4   name: letsencrypt-prod
5 spec:
6   acme:
7     server: https://acme-v02.api.letsencrypt.org/directory
8     email: admin@example.com

```

```

9   privateKeySecretRef:
10     name: letsencrypt-prod
11   solvers:
12     - http01:
13       ingress:
14         class: traefik

```

Đoạn mã 3.12 - Khai báo ClusterIssuer cho Let's Encrypt

Ingress của Keycloak và ứng dụng demo chỉ cần khai báo annotation tương ứng, hệ thống sẽ tự động lấy và gia hạn chứng chỉ mà không cần thao tác thủ công.

3.9 Quản lý bí mật và cấu trúc kho GitOps

Để có thể lưu bí mật an toàn trong Git, giải pháp Sealed Secrets được sử dụng: bí mật được mã hóa bằng khóa công khai của cụm và chỉ có thể giải mã bởi controller trong cụm. Đoạn mã 3.13 minh họa một SealedSecret.

```

1  apiVersion: bitnami.com/v1alpha1
2  kind: SealedSecret
3  metadata:
4    name: keycloak-secret
5    namespace: identity
6  spec:
7    encryptedData:
8      KC_DB_PASSWORD: AgBy3i40JSWK+PiTySY...

```

Đoạn mã 3.13 - Ví dụ một SealedSecret

Kho GitOps được tổ chức theo mô hình base/overlay để tái sử dụng cấu hình giữa các môi trường, như minh họa trong Đoạn mã 3.14.

```

1  gitops/
2  |-- apps/
3  | |-- keycloak/
4  | '-- demo-app/
5  |-- base/
6  | '-- keycloak/
7  '-- overlays/
8     |-- dev/
9     '-- prod/

```

Đoạn mã 3.14 - Cấu trúc kho GitOps

3.10 Quy trình triển khai từng bước

Quy trình triển khai từ mã nguồn tới môi trường chạy gồm các bước: (1) lập trình viên đẩy thay đổi lên Git; (2) CI chạy kiểm thử và build image; (3) image được đẩy lên registry; (4) manifest được cập nhật; (5) ArgoCD phát hiện và đồng bộ lên cụm; (6) Kubernetes thực hiện cuộn chiếu; (7) hệ thống giám sát ghi nhận trạng thái. Mỗi bước đều có thể kiểm tra và quay lui khi cần.

3.11 Kiểm thử khả năng tự phục hồi

Để kiểm chứng khả năng tự phục hồi, có thể xóa một Pod của Keycloak và quan sát Kubernetes tự tạo Pod mới. Bảng 3.2 ghi nhận kết quả kiểm thử theo chỉ số thời gian phục hồi trung bình (MTTR), đồng thời kiểm tra dịch vụ không gián đoạn nhờ các bản sao còn lại.

Bảng 3.2 - Kết quả kiểm thử tự phục hồi (khung ghi nhận)

Kịch bản	MTTR	Gián đoạn
Xóa một Pod Keycloak	(đo)	Không
Xóa Pod cơ sở dữ liệu	(đo)	Ngắn
Cuộn chiếu phiên bản mới	(đo)	Không

3.12 Triển khai cơ sở dữ liệu và bảo mật mạng

Cơ sở dữ liệu PostgreSQL được triển khai bằng StatefulSet để đảm bảo dữ liệu được lưu trên vùng nhớ ổn định (Đoạn mã 3.15). Song song đó, chính sách mạng (NetworkPolicy) được áp dụng để chỉ cho phép Keycloak kết nối tới cơ sở dữ liệu, giảm bề mặt tấn công (Đoạn mã 3.16).

```

1 apiVersion: apps/v1
2 kind: StatefulSet
3 metadata:
4   name: postgres
5   namespace: identity
6 spec:
7   serviceName: postgres
8   replicas: 1
9   selector:
10    matchLabels: { app: postgres }
11   template:
12     metadata:
13       labels: { app: postgres }

```

```

14 spec:
15   containers:
16     - name: postgres
17       image: postgres:16
18       ports:
19         - containerPort: 5432
20       envFrom:
21         - secretRef: { name: postgres-secret }
22       volumeMounts:
23         - name: data
24           mountPath: /var/lib/postgresql/data
25   volumeClaimTemplates:
26     - metadata: { name: data }
27     spec:
28       accessModes: ["ReadWriteOnce"]
29       resources:
30         requests: { storage: 5Gi }

```

Đoạn mã 3.15 - Manifest StatefulSet cho PostgreSQL

```

1 apiVersion: networking.k8s.io/v1
2 kind: NetworkPolicy
3 metadata:
4   name: allow-keycloak-to-postgres
5   namespace: identity
6 spec:
7   podSelector:
8     matchLabels: { app: postgres }
9   ingress:
10     - from:
11         - podSelector:
12             matchLabels: { app: keycloak }
13     ports:
14       - port: 5432

```

Đoạn mã 3.16 - Chính sách mạng giới hạn truy cập cơ sở dữ liệu

3.13 Kết chương

Chương này đã trình bày quá trình triển khai hệ thống từ môi trường cục bộ với Docker Compose, cấu hình định danh như mã nguồn, ứng dụng demo SSO, cho tới thiết kế triển khai trên Kubernetes cùng quy trình CI/CD và GitOps. Chương tiếp theo

đánh giá kết quả đạt được và phân tích các khía cạnh bảo mật.

Chương 4

ĐÁNH GIÁ KẾT QUẢ VÀ THẢO LUẬN

4.1 Phương pháp đánh giá

Hệ thống được đánh giá theo ba khía cạnh. Thứ nhất, về **chức năng**, các ca kiểm thử được thực hiện cho luồng đăng nhập SSO, cơ chế cấu hình realm tự động và xác thực đa yếu tố. Thứ hai, về **bảo mật**, các mối đe dọa được phân tích theo mô hình STRIDE và đối chiếu với biện pháp đã triển khai. Thứ ba, về **vận hành**, mức độ tự động hóa và khả năng tái lập của hệ thống được đánh giá. Việc kiểm thử được tiến hành trên môi trường triển khai cục bộ (Docker Compose) và môi trường cụm (k3s).

4.2 Kết quả kiểm thử chức năng

Bảng 4.1 trình bày các ca kiểm thử chức năng đã thực hiện cùng kết quả thu được.

Bảng 4.1 - Kết quả kiểm thử chức năng

Mã	Ca kiểm thử	Kết quả	Ghi nhận
TC1	Đăng nhập đúng thông tin	Đạt	Nhận token và truy cập ứng dụng
TC2	Đăng nhập sai mật khẩu	Đạt	Bị từ chối và có ghi log
TC3	Xác thực bằng mã TOTP	Đạt	Yêu cầu nhập đúng mã
TC4	Access token hết hạn	Đạt	Bị từ chối hoặc được làm mới
TC5	Áp dụng cấu hình realm từ JSON	Đạt	Realm được cập nhật theo mã nguồn
TC6	Chạy nhiều bản sao	Đạt	Các Pod cùng phục vụ, phiên ổn định

Kết quả cho thấy các chức năng cốt lõi của hệ thống hoạt động đúng như thiết kế. Bảng 4.2 đối chiếu kết quả này với các yêu cầu chức năng đã đặt ra ở Chương 2.

4.3 Đánh giá bảo mật

Các mối đe dọa được phân tích theo mô hình STRIDE và đối chiếu với biện pháp đã áp dụng trong hệ thống (Bảng 4.3).

Kết quả phân tích cho thấy việc kết hợp SSO tập trung, MFA, TLS và quản lý bí mật giúp giảm đáng kể bề mặt tấn công so với việc mỗi ứng dụng tự lưu trữ thông tin xác thực.

Bảng 4.2 - Đối chiếu yêu cầu chức năng và kết quả

Mã	Yêu cầu	Kết quả
FN1	Dịch vụ định danh tập trung với Keycloak	Đạt
FN2	Đăng nhập SSO vào ứng dụng demo qua OIDC	Đạt
FN3	Quản lý realm/client/role/user như mã nguồn	Đạt
FN4	Xác thực đa yếu tố (TOTP)	Đạt
FN5	Áp dụng cấu hình tự động qua CI/CD	Đạt
FN6	Ứng dụng demo hiển thị thông tin từ token	Đạt

Bảng 4.3 - Phân tích mối đe dọa theo STRIDE

Loại	Mối đe dọa	Biện pháp đã áp dụng
Spoofing	Giả mạo người dùng, đánh cắp phiên	MFA, thời hạn token ngắn, PKCE
Tampering	Sửa đổi token hoặc cấu hình	Chữ ký JWT, cấu hình quản lý qua Git
Repudiation	Phủ nhận hành động	Ghi log kiểm toán của Keycloak
Information disclosure	Lộ dữ liệu nhạy cảm	TLS, bí mật tách khỏi mã nguồn
Denial of service	Làm quá tải IdP	Giới hạn tần suất, chạy nhiều bản sao
Elevation of privilege	Leo thang đặc quyền	RBAC, đặc quyền tối thiểu, NetworkPolicy

4.4 Đánh giá mức độ tự động hóa

Bảng 4.4 tổng hợp mức độ tự động hóa đạt được đối với từng hạng mục của hệ thống.

Bảng 4.4 - Mức độ tự động hóa đạt được

Hạng mục	Mức độ	Hình thức
Xây dựng image	Đạt	Pipeline CI (GitHub Actions)
Quản lý cấu hình định danh	Đạt	Realm JSON và keycloak-config-cli
Triển khai lên cụm	Đạt	Helm và manifest Kubernetes
Đồng bộ theo GitOps	Đạt	ArgoCD
Đo lường và giám sát	Đạt	Điểm cuối /metrics của Keycloak
Quản lý bí mật	Chưa	Biến môi trường

4.5 Đánh giá hiệu năng

Để đánh giá sơ bộ khả năng đáp ứng của dịch vụ định danh, kịch bản sinh tải bằng k6 (trình bày ở Phụ lục A.5) được sử dụng để gửi yêu cầu cấp token tới Keycloak với mức tải tăng dần. Kết quả ghi nhận cho thấy dịch vụ phản hồi ổn định và không phát sinh lỗi trong suốt thời gian kiểm thử. Việc đo đạc định lượng chi tiết được xác định là nội dung tiếp tục của giai đoạn phát triển.

4.6 So sánh với phương án triển khai thủ công

Bảng 4.5 so sánh phương án triển khai tự động đã xây dựng với cách triển khai thủ công truyền thống.

Bảng 4.5 - So sánh triển khai thủ công và tự động

Tiêu chí	Thủ công	Tự động (đã xây dựng)
Tính tái lập	Thấp, phụ thuộc người thao tác	Cao, tái lập từ Git
Thời gian triển khai	Lâu, dễ sai sót	Nhanh, nhất quán
Truy vết thay đổi	Khó	Có, qua lịch sử Git
Khả năng khôi phục	Chậm	Đồng bộ lại từ Git
Yêu cầu kỹ năng	Thấp	Cao hơn (đầu tư ban đầu)

4.7 Thảo luận

Bảng 4.6 tổng hợp ưu điểm và hạn chế của hệ thống đã xây dựng.

Bảng 4.6 - Ưu điểm và hạn chế của hệ thống

Ưu điểm	Hạn chế
Xác thực tập trung, một nguồn định danh duy nhất	Phụ thuộc vào IdP (điểm tin cậy tập trung)
Cấu hình tái lập và truy vết qua Git	Đòi hỏi kỹ năng và công cụ hỗ trợ
Sẵn sàng chạy nhiều bản sao và tự phục hồi	Chưa được kiểm chứng ở quy mô lớn
Bảo mật theo lớp (TLS, MFA, RBAC)	Quản lý bí mật còn ở mức cơ bản

Việc kết hợp xác thực tập trung với triển khai tự động cho thấy đây là hướng tiếp cận phù hợp: dịch vụ định danh vừa được chuẩn hóa về giao thức, vừa được vận hành có kiểm soát và tái lập. Các hạn chế nêu trên là cơ sở cho những nội dung phát triển tiếp theo được trình bày ở phần Kết luận.

4.8 Kết chương

Chương này đã trình bày kết quả kiểm thử chức năng, kết quả đánh giá bảo mật theo mô hình STRIDE, mức độ tự động hóa đạt được, đánh giá hiệu năng sơ bộ và so sánh với phương án triển khai thử công.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Trong khuôn khổ chuyên đề, các kết quả chính đã đạt được bao gồm:

- Hệ thống hóa cơ sở lý thuyết về xác thực tập trung, giao thức OAuth 2.0 và OpenID Connect, cũng như các phương pháp triển khai tự động.
- Thiết kế kiến trúc ba tầng cho hệ thống định danh tập trung dựa trên Keycloak, với điểm nhấn là quản lý cấu hình định danh như mã nguồn.
- Triển khai thành công dịch vụ Keycloak cùng PostgreSQL và ứng dụng demo, minh họa luồng đăng nhập SSO qua OIDC với PKCE.
- Xây dựng được quy trình CI/CD và định hướng triển khai theo mô hình GitOps trên Kubernetes (k3s).
- Đánh giá hệ thống về chức năng và bảo mật, đề xuất khung đo lường hiệu năng cho giai đoạn tiếp theo.

Bảng 4.7 đối chiếu các mục tiêu đặt ra với kết quả đạt được.

Bảng 4.7 - Đối chiếu mục tiêu và kết quả đạt được

Mục tiêu	Kết quả	Mức độ
Xây dựng hệ thống định danh tập trung	Đã triển khai Keycloak, SSO OIDC	Đạt
Quản lý cấu hình như mã nguồn	Realm JSON và config-cli	Đạt
Tự động hóa triển khai	CI/CD, thiết kế GitOps	Đạt một phần
Đánh giá bảo mật	Phân tích STRIDE và biện pháp	Đạt

2. Hạn chế

Bên cạnh kết quả đạt được, chuyên đề còn một số hạn chế:

- Hệ thống mới được kiểm thử ở quy mô nhỏ, chưa đánh giá đầy đủ dưới tải cao và trong điều kiện nhiều bản sao.

- Một số hạng mục tự động hóa (IaC, GitOps, giám sát) mới dừng ở mức thiết kế, chưa hoàn thiện và vận hành ổn định.
- Chưa triển khai các cơ chế nâng cao như liên kết định danh (SAML/LDAP), zero-trust giữa các dịch vụ, hay đa vùng dự phòng.
- Phân tích hiệu năng và bảo mật còn ở mức định tính, cần bổ sung số liệu định lượng.

3. Hướng phát triển

Trên cơ sở các kết quả và hạn chế nêu trên, hướng phát triển thành đồ án tốt nghiệp bao gồm:

- **Hoàn thiện tự động hóa:** xây dựng đầy đủ IaC (Terraform, Ansible), GitOps hoàn chỉnh với ArgoCD và quản lý bí mật bằng Vault/Sealed Secrets.
- **Tăng cường tính sẵn sàng cao:** cấu hình nhiều bản sao Keycloak, sao chép PostgreSQL, kiểm thử khả năng tự phục hồi và cân bằng tải.
- **Đo lường định lượng:** sử dụng k6/Locust để đo độ trễ, thông lượng và MTTR; trực quan hóa bằng Grafana.
- **Mở rộng bảo mật:** triển khai zero-trust với mTLS giữa các dịch vụ, liên kết định danh với LDAP/Active Directory và đăng nhập WebAuthn.
- **Mở rộng quy mô:** nghiên cứu triển khai đa vùng và kịch bản dự phòng thảm họa (DR) với các chỉ tiêu RTO/RPO.

Tóm lại, chuyên đề đã xây dựng được nền tảng vững chắc cho một hệ thống xác thực tập trung có khả năng triển khai tự động, đồng thời mở ra nhiều hướng nghiên cứu và phát triển phù hợp với định hướng kỹ thuật hệ thống và bảo mật.

TÀI LIỆU THAM KHẢO

- [1] D. Hardt, *The OAuth 2.0 Authorization Framework*, RFC 6749, IETF, **october** 2012. **url:** <https://www.rfc-editor.org/rfc/rfc6749>
- [2] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros **and** C. Mortimore, *OpenID Connect Core 1.0*, OpenID Foundation, 2014. **url:** https://openid.net/specs/openid-connect-core-1_0.html
- [3] M. Jones, J. Bradley **and** N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, **may** 2015. **url:** <https://www.rfc-editor.org/rfc/rfc7519>
- [4] N. Sakimura, J. Bradley **and** N. Agarwal, *Proof Key for Code Exchange by OAuth Public Clients*, RFC 7636, IETF, **september** 2015. **url:** <https://www.rfc-editor.org/rfc/rfc7636>
- [5] Red Hat, *Keycloak Documentation*, <https://www.keycloak.org/documentation>, Truy cập: 2026.
- [6] The Kubernetes Authors, *Kubernetes Documentation*, <https://kubernetes.io/docs/>, Truy cập: 2026.
- [7] Argo Project, *Argo CD Documentation*, <https://argo-cd.readthedocs.io/>, Truy cập: 2026.
- [8] adorsys, *keycloak-config-cli: Configuration as Code for Keycloak*, <https://github.com/adorsys/keycloak-config-cli>, Truy cập: 2026.
- [9] Keycloak Community, *Keycloak Terraform Provider*, <https://registry.terraform.io/providers/keycloak/keycloak/>, Truy cập: 2026.

PHỤ LỤC

Phụ lục A. Tập cấu hình triển khai cục bộ

```
1 # Trien khai cuc bo: Keycloak + PostgreSQL (docker compose up -d)
2 services:
3   postgres:
4     image: postgres:16
5     environment:
6       POSTGRES_DB: keycloak
7       POSTGRES_USER: keycloak
8       POSTGRES_PASSWORD: ${DB_PASSWORD}
9     volumes:
10       - pgdata:/var/lib/postgresql/data
11     healthcheck:
12       test: ["CMD-SHELL", "pg_isready -U keycloak"]
13       interval: 10s
14       retries: 5
15
16   keycloak:
17     image: quay.io/keycloak/keycloak:26.0
18     command: ["start-dev", "--import-realm"]
19     environment:
20       KC_DB: postgres
21       KC_DB_URL: jdbc:postgresql://postgres:5432/keycloak
22       KC_DB_USERNAME: keycloak
23       KC_DB_PASSWORD: ${DB_PASSWORD}
24       KC_HOSTNAME: localhost
25       KC_METRICS_ENABLED: "true"
26       KC_HEALTH_ENABLED: "true"
27       KEYCLOAK_ADMIN: admin
28       KEYCLOAK_ADMIN_PASSWORD: ${KC_ADMIN_PASSWORD}
29     ports:
30       - "8080:8080"
31     volumes:
32       - ./realm:/opt/keycloak/data/import:ro
33     depends_on:
34       postgres:
```

```

35         condition: service_healthy
36
37 volumes:
38     pgdata:

```

Đoạn mã A.1 - Tập docker-compose.yml triển khai Keycloak và PostgreSQL

Phụ lục B. Cấu hình realm định danh

```

1  {
2    "realm": "demo",
3    "enabled": true,
4    "sslRequired": "external",
5    "accessTokenLifespan": 300,
6    "ssoSessionIdleTimeout": 1800,
7    "otpPolicyType": "totp",
8    "otpPolicyDigits": 6,
9    "passwordPolicy": "length(8) and upperCase(1) and lowerCase(1) and
10     digits(1)",
11    "clients": [
12      {
13        "clientId": "demo-app",
14        "protocol": "openid-connect",
15        "publicClient": false,
16        "standardFlowEnabled": true,
17        "redirectUri": ["http://localhost:3000/callback"],
18        "attributes": { "pkce.code.challenge.method": "S256" }
19      }
20    ],
21    "roles": {
22      "realm": [ { "name": "user" }, { "name": "admin" } ]
23    },
24    "users": [
25      {
26        "username": "alice",
27        "enabled": true,
28        "email": "alice@example.com",
29        "credentials": [
30          { "type": "password", "value": "ChangeMe123", "temporary": true }
31        ],
32        "realmRoles": [ "user" ]

```

```
33 ]
34 }
```

Đoạn mã A.2 - Tập demo-realm.json

Phụ lục C. Pipeline tích hợp liên tục

```
1 name: ci
2 on:
3   push:
4     branches: [ main ]
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - name: Build image
11        run: docker build -t ghcr.io/${{ github.repository }}:${{ github.
12              sha }} .
13      - name: Scan image
14        uses: aquasecurity/trivy-action@master
15        with:
16          image-ref: ghcr.io/${{ github.repository }}:${{ github.sha }}
17          severity: 'CRITICAL,HIGH'
```

Đoạn mã A.3 - Tập ci.yml (GitHub Actions)

Phụ lục D. Manifest triển khai trên Kubernetes

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: keycloak
5   namespace: identity
6 spec:
7   replicas: 2
8   selector:
9     matchLabels: { app: keycloak }
10  template:
11    metadata:
12      labels: { app: keycloak }
13    spec:
```

```

14     containers:
15       - name: keycloak
16         image: quay.io/keycloak/keycloak:26.0
17         args: ["start", "--optimized"]
18         envFrom:
19           - secretRef: { name: keycloak-secret }
20         ports:
21           - containerPort: 8080
22         readinessProbe:
23           httpGet: { path: /health/ready, port: 8080 }
24     ---
25     apiVersion: v1
26     kind: Service
27     metadata:
28       name: keycloak
29       namespace: identity
30     spec:
31       selector: { app: keycloak }
32       ports:
33         - port: 8080
34         targetPort: 8080

```

Đoạn mã A.4 - Deployment và Service cho Keycloak

Phụ lục E. Kịch bản đo hiệu năng

```

1 import http from 'k6/http';
2 import { check } from 'k6';
3
4 export const options = {
5   stages: [
6     { duration: '30s', target: 20 },
7     { duration: '1m', target: 50 },
8     { duration: '30s', target: 0 },
9   ],
10 };
11
12 export default function () {
13   const res = http.post(
14     'http://localhost:8080/realms/demo/protocol/openid-connect/token',
15     { grant_type: 'client_credentials',
16       client_id: 'demo-app',

```

```
17     client_secret: __ENV.CLIENT_SECRET }  
18 );  
19 check(res, { 'status 200': (r) => r.status === 200 });  
20 }
```

Đoạn mã A.5 - Kịch bản k6 đo khả năng cấp token